

Unity 生态集成

本页面介绍与 Timeline、Animator、Visual Scripting、Input System、Networking、Scriptable Audio Pipeline、Chunity (Chuck)、Maestro / MPTK 的集成功能。

命名空间：

集成	命名空间
Animator	jp.kshoji.unity.midi
Timeline	jp.kshoji.unity.midi.timeline
Visual Scripting	jp.kshoji.unity.midi.visualscripting
Input System	jp.kshoji.unity.midi.integrations.inputsystem
Networking	jp.kshoji.unity.midi.net
Scriptable Audio	jp.kshoji.unity.midi.scriptableaudio
Chunity	jp.kshoji.unity.midi.integrations.chunity
MPTK	jp.kshoji.unity.midi.mptk

位置：

集成	位置
Animator	Assets/MIDI/Scripts/Integrations/Animator/
Timeline	Assets/MIDI/Scripts/Integrations/Timeline/
Visual Scripting	Assets/MIDI/Scripts/Integrations/VisualScripting/
Input System	Assets/MIDI/Scripts/Integrations/InputSystem/
Networking	Assets/MIDI/Scripts/Integrations/Networking/
Scriptable Audio	Assets/MIDI/Scripts/Integrations/ScriptableAudio/
Chunity	Assets/MIDI/Scripts/Integrations/Chunity/
MPTK	Assets/MIDI/Scripts/Integrations/MPTK/

Unity-MCP（可选）

可通过 [Unity-MCP](#) 从 Cursor 等客户端做诊断与装配。工具位于 `Assets/MIDI/Scripts/Integrations/Mcp/`，仅在安装 `com.ivanmurzak.unity.mcp`（及 NuGet 门控）时编译。**不随 Asset Store 核心包分发。**

项	值
示例提示词	unity-mcp-sample-prompts.md
检查清单	unity-mcp-define-matrix.md
Integration README	../././Scripts/Integrations/Mcp/README.md
VST 主机	另包 — Unity-VST3-Bridge

最短路径：安装 Unity-MCP → Play Mode `midi-init` → `midi-devices-list`。已构建的 **Player** 需添加 `MidiMcpRuntimeBootstrap`，将 **Host** 指向局域网 MCP Server，并仅在 Editor 完成接线后使用控制工具（见 [MCP README](#)）。网络工具需 `FEATURE_MIDI_NETWORK`（`midi-net-*` 的 setup 属 Editor，`discovery-status` / `rtt` 属 Player）。MIDI 2.0 / MPE / CI 在核心 MCP 程序集（`midi2-*`、`mpe-zone-status`、`midi-ci-discover`）。VST 主机 MCP 面向桌面，见 [Unity-VST3-Bridge](#)（非 Android MCP 目标）。

前提条件

Timeline 集成

- 1. 通过 Unity Package Manager 安装 **Timeline** (`com.unity.timeline`)
- 2. 在 Project Settings 中添加脚本定义符号 `FEATURE_USE_TIMELINE`

项目	值
Assembly Definition	<code>jp.kshoji.midi.timeline</code>
脚本定义符号	<code>FEATURE_USE_TIMELINE</code>
设置路径	<code>Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols</code>

当该符号未定义时，Timeline 集成 `asmdef` 与样例代码 (`#if FEATURE_USE_TIMELINE` 块内) 不会被编译。

Visual Scripting 集成

- 1. 通过 Unity Package Manager 安装 **Visual Scripting** (`com.unity.visualscripting`)
- 2. 在 Project Settings 中添加脚本定义符号 `FEATURE_USE_VISUALSCRIPTING`

项目	值
Assembly Definition	<code>jp.kshoji.midi.visualscripting</code>
脚本定义符号	<code>FEATURE_USE_VISUALSCRIPTING</code>
设置路径	<code>Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols</code>

当该符号未定义时，Visual Scripting 集成 `asmdef` 与样例代码 (`#if FEATURE_USE_VISUALSCRIPTING` 块内) 不会被编译。

Input System 集成

- 1. 通过 Unity Package Manager 安装 **Input System** (`com.unity.inputsystem`)
- 2. 在 Project Settings 中添加脚本定义符号 `FEATURE_INPUT_SYSTEM`

项目	值
Assembly Definition	<code>jp.kshoji.midi.inputsystem</code>
脚本定义符号	<code>FEATURE_INPUT_SYSTEM</code>
设置路径	<code>Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols</code>

当该符号未定义时，Input System 集成 `asmdef` 与样例代码 (`#if FEATURE_INPUT_SYSTEM` 块内) 不会被编译。

Networking 集成

- 1. 添加 `FEATURE_MIDI_NETWORK` (UDP Hub/Client — 无需额外包)
- 2. 可选桥接 (包 **不同**捆) :
 - Mirror : 安装 → `FEATURE_MIRROR` (+ `MIRROR`)
 - Netcode : UPM `com.unity.netcode.gameobjects` → `FEATURE_NETCODE` (`MIDI_HAS_NETCODE`)
 - WNet2 : 客户端 + `WNet2.Runtime.asmdef` → `FEATURE_WNET2` (`MIDI_HAS_WNET2`) ; Lobby/Game 服务器需单独启动

项目	值
Core 程序集	jp.kshoji.midi.net
桥接	jp.kshoji.midi.net.mirror / .netcode / .wsnet2

详情：[Networking 集成](#) · [构建后处理](#)

Scriptable Audio Pipeline 集成

1. 使用 **Unity 6000.3 LTS** 或更高版本（WebGL 除外）
2. 在 Project Settings 中添加脚本定义符号 `FEATURE_SCRIPTABLE_AUDIO`
3. 可选包（使用本仓库时已随 `Packages/manifest.json` 一同附带）：
 - `com.unity.burst`
 - `com.unity.collections`

项目	值
Assembly Definition	jp.kshoji.midi.scriptableaudio
脚本定义符号	FEATURE_SCRIPTABLE_AUDIO
设置路径	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

当该符号未定义时，或在 Unity 6.2 及更早版本 / WebGL 上，集成 `asmdef` 不会被编译（`ScriptableAudioUtility.IsAvailable == false`）。核心 `jp.kshoji.midi` 仍可照常构建。

启用步骤的详细说明请参阅 [构建后处理 — Scriptable Audio Pipeline 集成](#)。

Chunity (Chuck) 集成

1. 另行安装 [Chunity](#)（本插件不附带运行时）
2. 将 `Integrations/Chunity/Optional/Chunity.Runtime.asmdef.example` 复制到 Chunity Scripts 根目录，命名为 `Chunity.Runtime.asmdef`
3. 在 Project Settings 中添加脚本定义符号 `FEATURE_CHUNITY`

项目	值
Assembly Definition	jp.kshoji.midi.chunity
脚本定义符号	FEATURE_CHUNITY
设置路径	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

当该符号未定义时，集成 `asmdef` 与样例的 `#if FEATURE_CHUNITY` 块不会被编译。启用步骤的详细说明请参阅 [构建后处理 — Chunity 集成](#)。

Scriptable Audio Generator（附加）

1. 将 [Chunity Optional 补丁说明](#) 中的 `useBuiltInAudioFilter` 补丁应用到 Chunity
2. 安装 Unity 包 `** com.unity.collections **`（`jp.kshoji.midi.chunity.scriptableaudio` 需要）
3. 添加 `FEATURE_CHUNITY_SCRIPTABLE_AUDIO`（需要 `FEATURE_CHUNITY`）

项目	值
Assembly Definition	jp.kshoji.midi.chunity.scriptableaudio
包	com.unity.collections
脚本定义符号	FEATURE_CHUNITY + FEATURE_CHUNITY_SCRIPTABLE_AUDIO
样例	ChunityGeneratorWorkflowSampleScene

详情：[Chunity 集成](#)

Maestro / MPTK 集成

1. 另行安装 Maestro / MidiPlayerTK（本插件不附带运行时）
2. 在 Project Settings 中添加脚本定义符号 `FEATURE_USE_MPTK`
3. 使用 **Maestro Pro** API 时，再额外添加 `MPTK_PRO`

项目	值
Assembly Definition	jp.kshoji.midi.mptk
脚本定义符号	FEATURE_USE_MPTK （+ 可选 MPTK_PRO）
位置	Assets/MIDI/Scripts/Integrations/MPTK/

`FEATURE_USE_MPTK` 启用 Free 级别桥接（作为虚拟 MIDI 设备的输出汇 / 将 MPTK 作为输入源的适配器）。
`MPTK_PRO` 进一步启用以 `#if MPTK_PRO` 守卫、依赖 Maestro Pro API 的代码，例如 `OnMidiEvent` rewrite 管线（`MptkMidiEventPipeline`）、`Writer / External` 播放器桥、`InnerLoop`、`ListPlayer`、`Spatializer` 及相关 Pro 示例。未定义 `MPTK_PRO` 时，这些 Pro 专用路径不会参与编译；仅启用 `FEATURE_USE_MPTK` 时仍可构建 Free 级别集成。

详情请参阅 [Maestro / MPTK 集成](#) 以及 [MPTK 集成](#)。

Animator 集成

- 无需额外的包或脚本定义符号（已包含在核心 asmdef `jp.kshoji.midi` 中）

详细的启用步骤请参阅 [构建后处理 — Unity 集成包](#)。

Animator 集成

将 MIDI 输入自动转换为 Unity Animator 的参数。演奏类、系统类消息通过绑定进行映射，SysEx / Raw UMP 则通过 `onMessage` 回调接收。

组件

组件	作用
<code>MidiAnimatorDriver</code>	MIDI 接收 → Animator 参数 + <code>onMessage</code>
<code>MidiAnimatorMapping</code>	ScriptableObject。可共享的绑定定义
<code>MidiBlendTreeDriver</code>	将 2-4 通道 CC 供给至 Blend Tree 轴

双层模型

层	对象	用途
绑定	可映射为 Float / Int / Bool / Trigger 的 MIDI	CC、音符、Program Change、Clock 等
<code>onMessage</code>	全部消息（含 SysEx / Raw UMP）	自定义脚本、payload 处理

`onMessage` 与绑定可对同一消息同时使用。若要避免重复处理，请只使用其中一种。

绑定（`MidiParameterBinding`）

字段	说明
<code>messageType</code>	<code>MidiOutgoingMessageType</code> （NoteOn / ControlChange / TimingClock 等）
<code>group</code>	0-15, <code>-1</code> = 所有 group
<code>channel</code>	0-15, <code>-1</code> = 所有通道（对于 Clock / Start 等 group 专用消息将被忽略）
<code>controllerOrNote</code>	音符 / CC / Program 编号等
<code>valueFilter</code>	值过滤, <code>-1</code> = 全部
<code>mapMessageValue</code>	将 NoteOn 的 velocity 映射为 Float/Int（相当于旧的 Velocity）
<code>animatorParameterName</code>	目标 Animator 参数名
<code>parameterType</code>	Float / Int / Bool / Trigger
<code>responseCurve</code>	归一化输入 0-1 → 输出
<code>smoothingTime</code>	CC / Pitch Bend 等的平滑秒数
<code>deviceIdFilter</code>	设备 ID 过滤（空 = 全部）

映射规则

messageType	推荐 parameterType	行为
NoteOn / NoteOff	Bool / Trigger	On/Off 或 Trigger（仅在 On 时）
NoteOn + mapMessageValue	Float / Int	velocity 归一化
ControlChange / PitchWheel / Aftertouch	Float	0-1 归一化 + 曲线
ProgramChange / SongSelect 等	Int	直接设置 number
TimingClock / Start / Stop / Reset 等	Trigger	接收时执行 SetTrigger
SystemExclusive / Raw UMP	—	仅 onMessage

注意：将 TimingClock 绑定到 Trigger 会在每个 Tick 触发。请注意 Animator Trigger 的消费时机，或在 onMessage 中进行抽稀处理。

设置

- 1. 将 MidiAnimatorDriver 挂载到带有 Animator 的 GameObject 上
- 2. 通过 Assets > Create > MIDI > Animator Mapping 创建映射（可选）
- 3. 配置绑定（Inspector 中的 Message Type / Group / Channel）
- 4. 若需要 SysEx / UMP，请将监听器连接到 onMessage
- 5. 在场景 bootstrap 中调用 MidiManager.InitializeMidi()

绑定示例

用途	messageType	controllerOrNote	parameterType	参数名
运动速度	ControlChange	1	Float	Speed
跳跃	NoteOn	60	Trigger	Jump
击键强度	NoteOn + mapMessageValue	60	Float	StrikePower
音色编号	ProgramChange	5	Int	Program
播放开始	Start	—	Trigger	TransportStart

onMessage 示例

```
driver.onMessage.AddListener(args =>
{
    if (args.message.messageType == MidiOutgoingMessageType.SystemExclusive)
    {
        var payload = args.message.payload;
        // SysEx 处理
    }
});
```

MidiBlendTreeDriver

面向 XY 手柄和 DJ 控制器用途。指定 2-4 个 CC，并将 0-1 归一化值供给至对应的 Animator float 参数（默认：BlendX / BlendY）。

Timeline 集成

在 Unity Timeline 上进行 SMF 播放、MIDI 记录和标记生成。

轨道与片段

类型	类	绑定	说明
播放轨道	MidiPlaybackTrack	SmfPlayer	将 SMF 片段同步至 Timeline 时间播放
播放片段	MidiPlaybackClip	—	MidiSequenceAsset 引用、BPM、通道重映射、mute / solo
记录轨道	MidiRecordTrack	MidiRecorder	记录片段区间内的 MIDI 输入
记录片段	MidiRecordClip	—	在片段激活期间 Start/Stop 记录

设置（播放）

1. 在 GameObject 上放置 PlayableDirector 和 SmfPlayer
2. 在 Timeline 窗口中选择 **Add > MIDI Playback Track**
3. 将 SmfPlayer 分配给轨道的 Binding
4. 在轨道上放置 **Midi Playback Clip**
5. 在片段的 Sequence Asset 中设置 MidiSequenceAsset
6. 在 Play 模式下确认 Timeline 与 SMF 同步播放

时间同步

Timeline 的片段本地时间是 SmfPlayer.Seek() 的主控。

- Play 期间：当差值超过 30ms 时通过 seek 进行跟随
- Pause / 拖拽：暂停 SmfPlayer 并 seek 到相应位置

可通过 MidiPlaybackClip 的 outputChannel （-1 = 保持原通道）、mute、solo 进行片段级别的输出控制。

限制：不支持在 Timeline 上进行钢琴卷帘编辑或直接编辑 UMP 片段。

标记

标记	说明
MidiMarker	在指定位置发送 MIDI 消息（需要 MidiTimelineNotificationReceiver）
MidiSignalEmitter	与 Timeline Signal 同时发送 MIDI 消息
MidiBarMarker	小节头。以标签显示拍号信息
MidiTempoMarker	速度变更位置

MidiMarker / MidiSignalEmitter 通过 MidiTimelineOutgoingMessage 设置发送内容。除 MIDI 1.0 演奏类、系统类消息外，还支持 SysEx / System Common / Raw UMP（MIDI 2.0）。

设置（发送标记）

1. 在与 PlayableDirector 相同的 GameObject 上添加 MidiTimelineNotificationReceiver（也可通过 Window > MIDI > Timeline > Add Notification Receiver To Director）
2. 在 Timeline 的 Marker Track 上放置 MidiMarker 或 MidiSignalEmitter

3. 在 Inspector 中设置 Message Type / Group / Channel / Payload (SysEx、UMP 时)

自动生成：在 Project 窗口中选择 `MidiSequenceAsset`，然后执行 `Window > MIDI > Timeline > Generate Markers From Sequence Asset`。请在 Timeline 窗口中打开 PlayableDirector 的状态下使用。

Signal 联动

将 `MidiTimelineSignalHandler` 设置为 Signal Receiver 的目标，即可从 Timeline Signal 调用以下方法：

- `SendNoteOn(int note, int velocity)`
- `SendNoteOff(int note, int velocity)`
- `SendControlChange(int controller, int value)`
- `SendProgramChange(int program)`
- `SendSystemExclusive(byte[] payload)`
- `SendRawUmp(uint[] umps)`
- `SendOutgoingMessage(MidiTimelineOutgoingMessage message)` — 支持所有消息类型

Visual Scripting 集成

无需编写代码即可在 Script Graph 上构建 MIDI 输入输出。

桥接

将 `MidiVisualScriptingBridge` 添加到挂载了 Script Graph 的 `GameObject` 上。它会将来自 `MidiManager` 的事件转发到 Visual Scripting 的 Event Bus。带类型的事件节点与通用的 `On MIDI Message` 可以同时使用，但若对同一消息连接两者会导致重复执行，因此请只使用其中一种。

事件节点（Events > MIDI）

节点	输出
On MIDI Message	deviceId、messageType、group、channel、number、value、data3、payload、payloadLength — 全部 MIDI 1.0 / SysEx / Raw UMP
On MIDI Note On	group、channel、note、velocity、deviceId
On MIDI Note Off	group、channel、note、velocity、deviceId
On MIDI Control Change	group、channel、controller、value、deviceId
On MIDI Program Change	group、channel、program、deviceId
On MIDI Pitch Bend	group、channel、amount (0–16383)、deviceId

Action 节点

类别	节点
MIDI > Send	Send MIDI Message （通用 DTO + payload）、Send MIDI Note On / Off / Control Change / Program Change / Pitch Bend （均支持 <code>group</code> ）
MIDI > Playback	SMF Player Play / Stop
MIDI > Utility	Get Active MIDI Notes、 Serialize MIDI UMP Words 、 Deserialize MIDI UMP Words
MIDI > Note	Note Name To Number / Number To Note Name / Is Note In Scale

Send MIDI Message 的 payload 示例

messageType	payload
SystemExclusive	SysEx 字节序列（ <code>F0 ... F7</code> ）
SystemCommonMessage	System Common 字节序列
Midi2RawUmp	以 big-endian 4 字节为单位连接的 UMP word 序列组成的字节序列。也可通过 <code>Serialize MIDI UMP Words</code> 单元生成

设置

- 在 Package Manager 中安装 Visual Scripting

2. 将 `Script Machine` 和 `MidiVisualScriptingBridge` 挂载到 `GameObject` 上
3. 创建 Script Graph, 并从 **Events > MIDI** 放置事件节点
4. 在场景 bootstrap 中调用 `MidiManager.InitializeMidi()`
5. 在 Play 模式下确认 MIDI 输入 → 图被执行

Is Note In Scale

使用 `MidiScaleUtility`。支持 `Major` / `NaturalMinor` / `MajorPentatonic` / `MinorPentatonic`。

Input System 集成

使用 Unity Input System 的 Synthetic Device 与 `.inputactions`，将 MIDI 输入输出接入现有基于 Input Action 的游戏逻辑。

组件

组件	方向	作用
MidiInputSystemBridge	MIDI → Input System	将接收到的 MIDI 注入 Synthetic Device。SysEx / Raw UMP 通过 <code>onMessage</code> 回调
InputSystemToMidiBridge	Input System → MIDI	将 Input Action 作为 <code>MidiOutgoingMessage</code> 发送
MidiSyntheticDevice	—	128 音符 + 128 CC + 通道专用轴
MidiInputSystemMapping	—	Action 名与 MIDI 条件的 ScriptableObject

Synthetic Device 布局

控件	路径示例	内容
<code>note[128]</code>	<code><MidiSynthetic>/note60</code>	音符 0–1（velocity / Poly AT）
<code>cc[128]</code>	<code><MidiSynthetic>/cc1</code>	CC 0–1
<code>pitch[16]</code>	<code><MidiSynthetic>/pitch0</code>	Pitch Bend 0–1（0.5 = 居中）
<code>program[16]</code>	<code><MidiSynthetic>/program0</code>	Program Change 0–1
<code>channelPressure[16]</code>	<code><MidiSynthetic>/channelPressure0</code>	Channel Aftertouch 0–1
<code>systemPulse[16]</code>	<code><MidiSynthetic>/systemPulse0</code>	Clock / Start / Stop 等的脉冲（group 索引）

SysEx / System Common / Raw UMP 无法映射到按钮/轴模型，因此 Synthetic Device 不支持。请通过 `MidiInputSystemBridge.onMessage` 以 `MidiOutgoingMessage` DTO 的形式接收。

映射（MIDI → Input System）

通过 `Assets > Create > MIDI > Input System > Mapping` 创建 `MidiInputSystemMapping`。

字段	说明
<code>actionName</code>	<code>.inputactions</code> 内的 Action 名（用于文档）
<code>messageType</code>	<code>MidiOutgoingMessageType</code> （ <code>NoteOn</code> / <code>ControlChange</code> / <code>PitchWheel</code> 等）
<code>group</code>	0–15， <code>-1</code> = 所有 group
<code>channel</code>	0–15， <code>-1</code> = 所有通道
<code>controllerOrNote</code>	音符 / CC 编号 / 匹配条件
<code>valueFilter</code>	值过滤， <code>-1</code> = 全部
<code>targetControlIndex</code>	CC 回退目标（ <code>-1</code> = <code>controllerOrNote</code> ）
<code>preferDedicatedControl</code>	若为 <code>true</code> 则使用 <code>pitch</code> / <code>program</code> / <code>channelPressure</code> / <code>systemPulse</code>
<code>invertAxis</code>	反转归一化轴

反向绑定（Input System → MIDI）

InputSystemToMidiBridge 的 InputToMidiBinding 采用通用 DTO 形式。

字段	说明
messageType / canceledMessageType	performed / canceled 时的发送类型
group / channel / number / value / data3	MidiOutgoingMessage 字段
useActionValue	将 Action 值转换为 0–127（Pitch 为 0–16383）
sendOnPerformed / sendOnCanceled	发送时机

旧字段 note / controller / sendControlChange 仍会被读取。

设置

- 1. 在 Package Manager 中安装 Input System
- 2. 定义 FEATURE_INPUT_SYSTEM
- 3. 在 GameObject 上添加 MidiInputSystemBridge，如有需要则分配 MidiInputSystemMapping
- 4. 在 .inputactions 的 Binding Path 中指定 <MidiSynthetic>/note60 等
- 5. 若需要输出，请添加 InputSystemToMidiBridge
- 6. 若需要 SysEx / UMP，请将监听器连接到 onMessage

样例

场景	路径
Input System 桥接	Assets/MIDI/Samples/Integrations/InputSystem/Scenes/InputSystemBridgeSampleScene.unity

Scriptable Audio Pipeline 集成

将 Unity 6.3+ 的 [Scriptable Audio Pipeline](#) 作为与 MIDI 时序联动的实时音频生成的**可选基础**提供。作为参考实现，包含节拍器 Generator、DSP 时钟桥接以及 Pipe 通信 DTO。

快速上手

- 1. 按照 [启用步骤](#) 定义 `FEATURE_SCRIPTABLE_AUDIO`
- 2. 在空的 `GameObject` 上添加 **Scriptable Audio Bootstrap** (`ScriptableAudioBootstrap`)
- 3. 进入 Play 模式 — 确认能听到 120 BPM 的咔嗒声

或打开样例场景：

```
Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioMetronomeSampleScene.unity
```

组件 / API

类型	作用
<code>ScriptableAudioBootstrap</code>	一次性接线 <code>AudioSource</code> + <code>Generator</code> + <code>Bridge</code>
<code>ScriptableAudioUtility</code>	<code>IsAvailable</code> 、 <code>EnsureAudioSource</code> 、 <code>AttachGenerator</code> 、 <code>ValidateSetup</code>
<code>MidiMetronomeGenerator</code>	参考 <code>IAudioGenerator</code> （节拍器咔嗒声）
<code>MidiDspClockBridge</code>	通过 DSP 快照 + Pipe 将 Transport 发送到 Generator
<code>MidiDspClockSnapshot</code>	面向游戏逻辑的只读时序 DTO
<code>MidiDspSequenceScheduler</code>	SMF tick → DSP 采样调度 + Pipe 音符事件
<code>MidiDspSequenceClockMode</code>	<code>SmfTempoMap</code> / <code>ExternalClock</code> （速度轴的互斥切换）
<code>MidiDspSequenceMidiOutBridge</code>	将已调度的音符镜像到 <code>MidiManager</code> （主线程发送）
<code>MidiDspUmpSequenceScheduler</code>	UMP 片段 tick → DSP 采样调度 + Pipe
<code>MidiDspUmpSequenceClockMode</code>	<code>UmpTempoMap</code> / <code>ExternalClock</code>
<code>MidiDspUmpMidi2OutBridge</code>	将已调度的 UMP 数据包镜像到 <code>MidiManager.SendMidi2RawUmp</code> （帧粒度）
<code>MidiDspUmpSequenceBootstrap</code>	一次性接线 <code>AudioSource</code> + UMP Scheduler + 参考合成器 + Clock
<code>UmpSequenceSynthGenerator</code>	参考 <code>IAudioGenerator</code> （UMP 复音，用于验证）
<code>UmpSequenceAsset</code>	保存 <code>.midi2</code> 二进制的 <code>ScriptableObject</code> （ <code>ToUmpSequence()</code> ）
<code>MidiDspUmpSequenceSmfFallback</code>	通过 <code>SequenceConverter</code> 委托给 SMF Scheduler 的回退
<code>MptkDspUmpSequenceOutput</code>	UMP Out → MPTK 虚拟设备（ <code>FEATURE_USE_MPTK</code> ；位于 <code>MPTK/ScriptableAudio/</code> ）
<code>ScriptableAudioSetupReport</code>	设置诊断（Error / Warning）

样例演示

模式	内容
独立	以 <code>MidiDspClockBridge.fallbackBpm</code> （默认 120）发出咔嗒声
外部 Clock	通过样例 UI 切换。从虚拟设备 <code>virtual:scriptable-audio-sample</code> 注入 Start / Timing Clock / Stop

SMF 序列播放（DSP 同步）

`MidiDspSequenceScheduler` 将 SMF 的 Note On/Off 映射到 Unity DSP 时钟上的绝对采样位置，并用参考合成器 `MidiSequenceSynthGenerator` 播放。它与 `SmfPlayer`（帧驱动、MIDI 设备输出）属于不同的层。

模式	内容
SMF 速度映射	本地 Transport（Play/Stop/Seek）。速度来自 SMF 内的元事件 + <code>tempoFactor</code>
外部 Clock	<code>clockMode = ExternalClock + MidiClockSync</code> 。跟随 Start/Stop，BPM 通过 <code>EstimatedBpm / externalClockReferenceBpm</code> 进行倍率换算
硬件 MIDI 输出	可选添加 <code>MidiDspSequenceMidiOutBridge</code> 。将与 Pipe 相同的调度镜像到 <code>MidiManager</code> （帧粒度）

样例：`Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioSequenceSampleScene.unity`

两者均与 `MidiManager.InitializeMidi()` 共存。

UMP 序列播放（DSP 同步）

`MidiDspUmpSequenceScheduler` 将 `UmpSequence`（.midi2 片段）的 Note On/Off 以及 UMP 通道声音映射到 Unity DSP 时钟上的绝对采样位置，并用参考合成器 `UmpSequenceSynthGenerator` 播放。它与 `UmpSequencer`（墙钟 + 专用线程驱动）属于不同的层。

模式	内容
UMP 速度映射	本地 Transport（Play / Pause / Stop / Seek）。速度来自 Flex <code>Set Tempo</code> + PPQ + <code>tempoFactor</code>
外部 Clock	<code>clockMode = ExternalClock + MidiClockSync</code> 。跟随 Start/Stop，BPM 通过 <code>EstimatedBpm / externalClockReferenceBpm</code> 进行倍率换算
硬件 MIDI 2.0 输出	可选添加 <code>MidiDspUmpMidi2OutBridge</code> 。将与 Pipe 相同的调度作为原始 UMP 数据包镜像到 <code>MidiManager.SendMidi2RawUmp</code> （帧粒度）
System / SysEx	设置 <code>scheduleSystemMessages = true</code> 可调度 UMP System（type 0x1）。Data SysEx（type 0x3）无论完成型还是分割重组型，均支持 Pipe（内联 ≤52B）及 <code>UmpOut</code> 镜像

使用区分：

用途	推荐
片段编辑 / 录制 / .midi2 试播	<code>UmpSequencer</code>
游戏内 BGM / 循环 / Seek / DSP 同步音频	<code>MidiDspUmpSequenceScheduler</code>
仅 MIDI 1.0 SMF	<code>MidiDspSequenceScheduler</code>
硬件 MIDI 2.0 输出（可接受帧粒度）	<code>MidiDspUmpMidi2OutBridge + MidiManager.SendMidi2RawUmp</code>

样例：`Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioUmpSequenceSampleScene.unity`

与 `MidiManager.InitializeMidi()` / `InitializeMidi2()` 共存。使用 UMP 输出桥接时需调用 `InitializeMidi2()`，并且必须已连接 MIDI 2.0 输出设备。

快速上手（UMP）

- 1. 按照 [启用步骤](#) 定义 `FEATURE_SCRIPTABLE_AUDIO`
- 2. 在空的 `GameObject` 上添加 **Midi Dsp Ump Sequence Bootstrap**（`MidiDspUmpSequenceBootstrap`）
- 3. 通过代码分配 `UmpSequence`，或使用样例场景的演示工厂
- 4. 进入 Play 模式 — 确认能通过参考合成器听到音符

```
var bootstrap = gameObject.AddComponent<MidiDspUmpSequenceBootstrap>();
bootstrap.sequence = UmpSequenceReader.ReadSequence(stream)[0];
bootstrap.EnsureReady();
bootstrap.Scheduler.Play();
```

若要将 `.midi2` 制作成资源，请创建 `UmpSequenceAsset`（菜单 **Assets > Create > MIDI > Import UMP Sequence Asset From File**，或 `CreateAssetMenu`），并将 `asset.ToUmpSequence()` 的结果传给 `Bootstrap / Scheduler`。详情请参阅 [SMF 工具 — UmpSequenceAsset](#)。

可在样例 UI 中确认的项目

UI	内容
C major scale (MIDI 1.0 / MIDI 2.0)	Note On/Off 播放（MIDI 2.0 为 32-bit velocity）
速度变更（120 → 90 BPM）	Flex Set Tempo 区间无拍子偏移
Loop first 4 notes	loopStartTick / loopEndTick（0–1920）
Tempo factor	播放速度倍率
External MIDI Clock	跟随 Start/Stop
Mirror UMP packets	启用 <code>MidiDspUmpMidi2OutBridge</code> （切换后桥接会自动创建）
Validate Setup	相当于 <code>ScriptableAudioUtility.ValidateUmpSequenceSetup</code> 的诊断日志

扩展功能

功能	用法
Group Mute / Solo	<code>MidiDspUmpSequenceScheduler.SetGroupMute(group, mute)</code> / <code>SetGroupSolo(group, solo)</code> （Group 0–15）
MIDI 2.0 控制 / 属性	<code>scheduleMidi2Controls</code> / <code>scheduleNativeUmpPackets</code> （以 Pipe 或原始 UMP 发送 CC / PC / PNC 等）
SMF 回退	Bootstrap 的 <code>playbackMode = SmfDelegated</code> 。 <code>SequenceConverter.ConvertUmpSequence</code> → <code>MidiDspSequenceScheduler</code>
MPTK 联动	MPTK/ScriptableAudio/ 中的 <code>MptkDspUmpSequenceOutput</code> （ <code>FEATURE_USE_MPTK</code> + <code>FEATURE_SCRIPTABLE_AUDIO</code> ）。将 UMP Out 路由到 MPTK 虚拟设备

已知限制（参考合成器 / Pipe）

项目	表现
System（type 0x1）	可发送到 Scheduler → Pipe / UmpOut。参考合成器不处理（不发声）
Data 128bit（type 0x5）及 3-word 以上	umpPacketTable + UmpOut 完整镜像。Pipe 内联及参考合成器仅支持 1–2 word
UMP Stream（type 0xF）	不在支持范围
硬件输出的精度	帧粒度（与内置合成器的 DSP 采样精度不同）
WebGL / Unity 6.2 及更早	不支持集成（同下方“限制”）

手动确认清单（样例场景）

在 Play Mode 下打开 ScriptableAudioUmpSequenceSampleScene，并确认以下内容。

- 1. MIDI 1.0 / MIDI 2.0 音阶通过参考合成器发声
- 2. 在速度变更演示中拍子不会偏移并累积
- 3. 循环 ON 时 4 音区间反复播放 / Seek 后也能重新开始
- 4. External Clock ON 时跟随虚拟设备的 Start/Stop
- 5. Validate Setup 无 Error，且 UMP Out OFF 时不出现桥接缺失 Warning
- 6. （可选）UMP Out ON + InitializeMidi2() + 实体设备时，原始 UMP 能送达
- 7. （可选）通过 UmpSequenceAsset 播放真实 .midi2，听感与 UmpSequencer 大体一致

Edit Mode 的自检（Pipe blittable、tick ↔ DSP、Extractor、风险应对）包含在 Window > MIDI > Validate Scriptable Audio Setup 或 ScriptableAudioUtility.ValidateFoundation() 中。尚未添加基于 Unity Test Runner 的自动回归。

有关实现的位置和诊断 API 的详情，也请参阅 Scriptable Audio 集成。

诊断

- 代码：ScriptableAudioUtility.ValidateSetup(gameObject) → ScriptableAudioSetupReport.ToSummary()
- 仅 UMP：ScriptableAudioUtility.ValidateUmpSequenceSetup(gameObject)
- 编辑器：Window > MIDI > Validate Scriptable Audio Setup（验证选中的 GameObject，未选中时验证场景中的第一个 Bootstrap。自动判别节拍器 / SMF / UMP）

限制

- WebGL：排除集成 asmdef。不会产生构建错误，API 为 IsAvailable == false
- Unity 6.2 及更早版本：因源码守卫而不编译
- 对同一输出的双重驱动：不要对同一乐曲输出同时使用 UmpSequencer 与 MidiDspUmpSequenceScheduler（有意为之的情况除外）
- 详情：平台 — Scriptable Audio / Scriptable Audio 集成

Chunity (Chuck) 集成

一项**可选集成**，将外部引入的 [Chunity](#) 上的 Chuck 补丁与本插件的 MIDI 输入输出、SMF、Clock、Timeline / Visual Scripting 连接起来。Chuck 内置的 `MidiIn` 依赖 OS 设备编号，不会经过本插件的虚拟设备、过滤器或网络路径，因此集成的核心是 **Unity MIDI → Chuck 全局变量 / Event**。

官方 API 参考：[Chunity Documentation](#)

快速上手

1. 按照 [启用步骤](#) 准备 Chunity 与 `FEATURE_CHUNITY`
2. 放置 `ChuckMainInstance` （可选 `ChuckSubInstance` ）
3. 在同一 GameObject（或引用目标）上为 `MidiChuckPatchHost` 分配 `.ck` / 内联补丁
4. 添加 `MidiChuckBridge` — 默认开启 **Use Default Convention**
5. 在 Play 模式下发送 MIDI（实体设备或虚拟 Inject），确认补丁有响应

组件

组件	作用	Phase
MidiChuckInstanceTarget	Main / Sub 派发 (String / 数组 / 关联数组 / RunFile args / ListenOnce 等)	A
MidiChuckBridge	MIDI → SetInt / SetFloat / Array / Associative + SignalEvent	1, A
MidiChuckPatchHost	执行内联 / TextAsset / StreamingAssets 的 .ck (Context Menu "Run Patch Now")	1
MidiChuckMapping	MIDI 条件 → ChuckK 全局 / Event (含 AssociativeInt / AssociativeFloat)	1, A
MidiChuckUtility	默认全局名与 IsAvailable / SetupHint / SetChuckLogLevel	1, E
MidiChuckEventToMidi	Chuck Event → UnityEvent / MIDI OUT (标量 Get + 数组读取 arraySource)	2, D
MidiChuckSmfLink	SmfPlayer → 虚拟设备 (默认 virtual:chuck-smf) → Bridge	2
MidiChuckClockSync	MidiClockSync → bpm / beat / bar / transport Event	2
MidiChuckPolyVoiceHelper	复音 voiceId (+ 可选 activeNotes[] 等)	2
MidiChuckFloatSyncer / MidiChuckIntSyncer / MidiChuckStringSyncer	Chuck*Syncer 包装 (从 Inspector 指定全局名)	B
MidiChuckFloatArraySyncer / MidiChuckIntArraySyncer	数组 Syncer 包装	B
MidiChuckParameterPoller	ParameterBinder 的读取对 (多参数轮询)	B
MidiChuckHostAdvancer	Unity 主控 timeStep / pos / tick Event 桥接	C
MidiChuckPatchLifecycle	协作式 stop Event + shred cleanup + 可选 Restart	C
MidiChuckSampleBank	Program / Note / Bank → 样本路径 → SetString + play Event	D
MidiChuckPreset	float/int 快照 (+ 可选补丁)	3
MidiChuckParameterBinder	预设应用、实时 SetFloat / SetInt	3
MidiChuckMarker 等	Timeline 标记 / 参数片段 (含 Phase E 动作)	3, E (FEATURE_USE_TIMELINE)
Chuck VS Units	Set/Get String、数组、RunFile、Event listen、HostAdvancer、PatchLifecycle、SampleBank 等	3, E (FEATURE_USE_VISUALSCRIPTING)

默认规约（映射未匹配时）

MIDI	Chuck
Note On	<code>midiNote</code> , <code>midiVelocity</code> , Event <code>noteOn</code>
Note Off	<code>midiNote</code> , Event <code>noteOff</code>
CC	<code>ccNumber</code> , <code>ccValue</code> , Event <code>controlChange</code> （可选 <code>cc[]</code> ）
Pitch Bend	<code>pitchBend</code> （-1..1）
Program Change	<code>program</code>

样例 `.ck` 一侧的最小契约示例：

```
global Event noteOn;
global Event noteOff;
global int midiNote;
global float midiVelocity;

SinOsc osc => dac;
while (true) {
    noteOn => now;
    Std.mtof(midiNote) => osc.freq;
    midiVelocity => osc.gain;
}
```

映射（`MidiChuckMapping`）

通过 `MidiChuckBinding` 声明“哪个 MIDI 条件对应哪个 Chuck 变量 / Event”。将 `messageType` / `group` / `channel` / `controllerOrNote`（均可 `-1` = 全部）与写入目标（`int` / `float` / `Event` / `float 数组`）、`floatScale`、`broadcastEvent` 组合。无需修改代码即可替换为其他 `.ck`。

Phase 2–3 的使用区分

用途	推荐
SMF → Chuck	用 <code>MidiChuckSmfLink</code> 统一虚拟设备，并使 <code>SmfPlayer.outputDeviceId</code> 与 Bridge 的 <code>deviceIdFilter</code> 一致
外部 Clock	<code>MidiChuckClockSync</code> （BPM → <code>bpm</code> ，拍 / 小节 / Start-Stop Event）
复音	<code>MidiChuckPolyVoiceHelper</code> + Bridge 的 <code>useDefaultConvention = false</code> （防止与单音规约重复触发）
预设 UI	<code>MidiChuckPreset</code> + <code>MidiChuckParameterBinder</code> （样例： <code>ChunityPresetsSampleScene</code> ）
Chuck → MIDI OUT	<code>MidiChuckEventToMidi</code>

Timeline / Visual Scripting （Phase 3 / E）

附加符号	Assembly	内容
FEATURE_USE_TIMELINE	jp.kshoji.midi.chunity.timeline	MidiChuckMarker （SetString / SetArray / RunFile / HostAdvancer / Lifecycle* / SampleBank 等） / MidiChuckTimelineNotificationReceiver / MidiChuckParamTrack
FEATURE_USE_VISUALSCRIPTING	jp.kshoji.midi.chunity.visualscripting	MIDI / Chunity 类别的单元（Phase E：Get* / RunFile / Event listen / HostAdvancer / PatchLifecycle / SampleBank 等）。注册： Window → MIDI → Visual Scripting → Register Chunity Nodes

Editor 诊断（无需附加 define）：**Window** → **MIDI** → **Chunity** → **Diagnostics** — 日志级别与集成状态。

Scriptable Audio Generator （附加选项）

在保持控制路径（Bridge / Mapping / Preset 等）不变的前提下，仅将音频输出从 OnAudioFilterRead 替换为 Unity 6.3+ 的 **Scriptable Generator**（IAudioGenerator）。它与节拍器等本体 Scriptable Audio 集成（FEATURE_SCRIPTABLE_AUDIO）属于不同的符号。

类型	作用
ChuckMainGeneratorDriver	用 AudioSource.generator 驱动 Main VM
ChuckSubGeneratorDriver	用 Generator 驱动 Sub（带空间化）
ChuckAudioOutputMode	Auto / FilterRead / ScriptableGenerator

模式	表现
Auto （推荐）	在支持的环境中使用 Generator，其他（WebGL 等）使用传统 FilterRead
FilterRead	与标准 Chunity 相同的 OnAudioFilterRead
ScriptableGenerator	严格使用 Generator（不支持时输出一次警告后回退）

设置：

- 应用 [启用步骤（Scriptable Audio Generator）](#) 的补丁、安装 `** com.unity.collections **`，并添加 FEATURE_CHUNITY_SCRIPTABLE_AUDIO
- 在 ChuckMainInstance 上添加 ChuckMainGeneratorDriver（使用 Sub 时在各 ChuckSubInstance 上添加 ChuckSubGeneratorDriver）
- 设置 **Audio Output Mode**（通常为 Auto）
- Play — Driver 在 Generator 连接期间设 useBuiltInAudioFilter = false，以防止重复播放

运行注意事项：

项目	内容
禁止重复播放	Generator 启用时禁用 FilterRead。不要同时驱动
Main → Sub 顺序	两者均为 Generator 时 ，ChuckGeneratorTracker 会在 Main 的 VM 推进后才允许 Sub。Main 为 FilterRead 时则依赖 Chunity Mixer 顺序
麦克风 (adc)	Main Driver 从主线程通过 lock-free 环形缓冲将录音缓冲供给至 DSP
静音	尊重 AudioSource.mute 。推进 VM 时钟的同时仅将输出静音
采样率	当 Chuck 初始率与 Generator AudioFormat.sampleRate 不同时发出警告

限制 / 路线图

- Chunity 运行时**不附带**。需要三者齐备：安装、Chunity.Runtime.asmdef 以及 FEATURE_CHUNITY
- 仅启用符号但未引入 Chunity 时会出现类型解析错误（与 MPTK 相同）
- 使用者始终可以直接调用 Chunity 原生 API。本插件是 MIDI 领域的薄桥接层
- **未实现（路线图）**：MIDI 2.0 / UMP 映射、InstanceTarget 的 String / Once / GetArray / RunFile 参数、Syncer / 关联数组 / *_AT 写入、VM / shred 控制辅助、UGen 探测、Host Time Advancer 等 — 参阅 [目录中的未来功能](#)
- 详细步骤：[Chunity 集成](#) / [构建后处理](#)

Networking 集成

LAN 上的 MIDI 事件 / SMF 播放同步。主线为 UDP（`FEATURE_MIDI_NETWORK`）。可选在 Mirror、Netcode 或 [WSNet2](#) 上承载相同 Codec。

组件	作用
<code>MidiNetworkHub</code> / <code>MidiNetworkClient</code>	UDP 主线
<code>MidiMirrorBridge</code> / <code>MidiNetcodeBridge</code> / <code>MidiWsnet2Bridge</code>	各传输上的 Host/Master 权威桥接

步骤：[Networking 集成](#)

示例场景

场景	路径
Animator 集成	Assets/MIDI/Samples/Integrations/Scenes/MidiAnimatorIntegrationSampleScene.unity
Timeline 集成	Assets/MIDI/Samples/Integrations/Scenes/MidiTimelineIntegrationSampleScene.unity
Visual Scripting 集成	Assets/MIDI/Samples/Integrations/Scenes/MidiVisualScriptingIntegrationSampleScene.unity
Input System 桥接	Assets/MIDI/Samples/Integrations/InputSystem/Scenes/InputSystemBridgeSampleScene.unity
网络 MIDI (UDP)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiNetworkJamSampleScene.unity
网络 MIDI (Mirror)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiMirrorNetworkSampleScene.unity
网络 MIDI (Netcode)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiNetcodeNetworkSampleScene.unity
网络 MIDI (WSNet2)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiWsnet2NetworkSampleScene.unity
Scriptable Audio 集成	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioMetronomeSampleScene.unity
Scriptable Audio SMF 序列	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioSequenceSampleScene.unity
Scriptable Audio UMP 序列	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioUmpSequenceSampleScene.unity
Chunity 桥接	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityBridgeSampleScene.unity
Chunity 工作流	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityWorkflowsSampleScene.unity
Chunity 预设	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityPresetsSampleScene.unity
Chunity 麦克风 FX	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityMicFxSampleScene.unity
Chunity Generator Workflow	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityGeneratorWorkflowSampleScene.unity

每个场景都支持通过虚拟 MIDI 输入进行 IMGUI 测试。详细步骤请参阅 [示例项目](#)。

Animator 用的 `MidiAnimatorSample.controller` 已随 `Samples/Integrations/Resources/` 一同附带。若要重新生成，请执行 `Window > MIDI > Samples > Generate Integration Sample Assets`。

VST3 插件宿主（独立软件包）

VST3 乐器/效果器宿主功能 不属于 本 MIDI 插件，MIDI 发行物中 不包含 `VstHostNative.dll` 、VST3 SDK 或 VST 宿主 C#。

请使用独立软件包：

项目	值
显示名称	Unity Plugin Host for VST3
UPM / Git URL	<code>https://github.com/kshoji/Unity-VST3-Bridge.git</code>
软件包 id	<code>jp.kshoji.unity.vst3nativehost</code>

安装、扫描路径与可选 MIDI 接线见该仓库文档（`Documentation~/usage.md` 等）。本 MIDI 包仅供订阅公开 MIDI 事件；请勿在 `Assets/MIDI` 下放置 VST 实现。桌面 Player 的 VST MCP 在 Bridge 侧；**Android 设备 MCP 不是 VST3 目标**。

VST 包中的可选 MIDI 辅助功能（需要 `FEATURE_MIDI_PLUGIN`）：

功能	组件
CC / 弯音 → VST 参数	<code>VstMidiParameterMapping</code> , <code>VstHostMidiParameterMapper</code>
SMF → VST 音源	<code>VstHostSmfLink</code> + <code>SmfPlayer.outputDeviceId</code> → 虚拟设备 → <code>VstHostMidiAdapter</code> （可按通道分流）
预设 / A/B	<code>VstPresetAsset</code> , <code>VstPresetBrowser</code> , Window → VST3 Host → Preset Browser

其他 VST 包集成（安装 Timeline / Input System 后可选程序集自动启用）：

功能	组件
Timeline 参数自动化	<code>VstParameterTrack</code> / <code>VstParameterClip</code> , <code>VstProgramChangeMarker</code> (<code>FEATURE_USE_TIMELINE</code>)
Animator → VST	<code>VstAnimatorMapping</code> , <code>VstAnimatorDriver</code>
Input System → VST	<code>InputSystemToVstBridge</code> (<code>FEATURE_INPUT_SYSTEM</code>) 或 MIDI 的 <code>InputSystemToMidiBridge</code> → Adapter
编辑器工具	Plugin Browser（分类 + Vendor/Tag）, Activity Monitor, Virtual Controller, Project Settings → VST3 Host
插件链	<code>VstPluginChain</code> , <code>VstHostChannelRouteSync</code> , <code>VstHostMidiFilterLink</code> , <code>VstHostEventSink</code>
Scriptable Audio（Unity 6.3+）	<code>VstHostGenerator</code> 、 <code>VstHostDspMidiOutBridge</code> → 调度器的 <code>extraTimedMidiOutput</code>
Visual Scripting	VST3 Host 单元 / 事件 (<code>FEATURE_USE_VISUALSCRIPTING</code>)
网络 MIDI → VST	<code>VstHostNetworkMidiLink</code> (<code>FEATURE_MIDI_NETWORK</code>)
Chunity → VST	<code>VstHostChuckEventMidiLink</code> 、 <code>VstHostChuckEffectBridge</code> (<code>FEATURE_CHUNITY</code>)

详情：VST 包 `Documentation~/timeline.md` / `animator-input.md` / `editor-tools.md` / `plugin-chain.md` / `scriptable-audio.md` / `visual-scripting.md` / `network-midi.md` / `chunity.md` 。

相关文档

- [面向游戏玩法的组件](#) — `SmfPlayer` / `MidiRecorder` / `MidiInputRouter` / `MidiClockSync`
- [SMF 工具](#) — `MidiSequenceAsset` / `UmpSequenceAsset` / `TempoMapExtractor`
- [实用工具](#) — `MidiNoteUtility` / `MidiMessageBuilder`
- [构建后处理](#) — 启用可选集成符号
- [示例项目](#) — 集成示例场景的步骤
- [嵌入的第三方模块](#) — 内置模块（VST 宿主为独立软件包，不同捆）